

From PySpark to ONNX: A Bit-Exact, JVM-Free Serving Path for an Edge Intrusion Detection System on Jetson Orin Nano Super

Thai Nguyen Vu¹, Dung Van Bui^{2,3}, Nhut Tri Do^{2,3}, and Van Du Nguyen⁴

¹ Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

thaivn_ph@utc.edu.vn

² Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam

³ University of Information Technology, Ho Chi Minh City, Vietnam

dungbv.17@grad.uit.edu.vn, trinhutdo@uit.edu.vn

⁴ Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

nvdu@hcmuaf.edu.vn

Abstract A companion study of ours deployed a leakage-aware Random Forest intrusion detector as a PySpark PipelineModel on a two-node Jetson Orin Nano Super cluster, for consistency with a Spark-based training phase, and measured that this choice is expensive: on the same forest and the same 29-feature input, a single-node microbenchmark showed scikit-learn reaching $9\times$ the Spark throughput and ONNX Runtime reaching $348\times$, at a fraction of the energy per inference. That paper left “wiring an ONNX session into the streaming pipeline” as future work. This paper does that. We introduce a swappable classifier-tier contract (spark | numpy | onnx) behind one engine-agnostic feature-matrix builder, so the Kafka topology, the anomaly gate, and the monitoring stack are unchanged. The ONNX backend is produced by reading the deployed model’s saved Parquet node tables directly and emitting a single TreeEnsembleClassifier graph, with no scikit-learn or retraining step in between, so the exported graph is the leakage-aware model validated offline, not a re-fit approximation of it. A pure-NumPy backend flattens the same forest into a node table for boards where no onnxruntime wheel matches the installed JetPack/Python combination. Both backends are selected with a single EDGE_ENGINE environment variable, requiring no change to the anomaly gate or the streaming roles. *[We report end-to-end Kafka throughput, latency, and energy per verdict for the three backends on the two-node Jetson cluster, and the prediction-agreement rate between each exported backend and the Spark model it was exported from, once the branch is benchmarked on hardware.]*

Keywords: Intrusion detection · Edge computing · ONNX Runtime · Model serving · Jetson Orin Nano Super · Apache Spark.

1 Introduction

Streaming ML-based intrusion detection increasingly runs at the network edge to cut bandwidth and response time [15, 5], and a companion study of ours deployed such a

system: a two-stage pipeline on two NVIDIA Jetson Orin Nano Super boards, with a lightweight autoencoder gate filtering benign flows before a PySpark `PipelineModel` classifies the remainder. That system reused the same Apache Spark stack [16, 10] for training (on a Spark standalone cluster spanning the two Jetson boards) and for serving (on the classifier board alone), on the reasoning that one stack is simpler to operate than two. The companion paper also measured the price of that simplicity: with the same Random Forest [2] and the same 29-feature SHAP-selected input, a single-node microbenchmark on the classifier board showed the JVM-hosted Spark path reaching only 22.6 flows/s, against 204.4 flows/s for the same forest under scikit-learn and 7,870.8 flows/s under ONNX Runtime, at a p95 latency of 532 ms, 49.8 ms, and 1.1 ms respectively. On an 8 GB ARM board with no swap headroom to spare, that JVM overhead is not a rounding error: it is the largest single knob left on the table after the anomaly gate has already removed most of the benign traffic.

The companion paper stopped at the microbenchmark and named the natural next step: export the deployed model to ONNX (or TensorRT) and wire the resulting session into the streaming pipeline, without disturbing the parts of the system that already work – the Kafka topology, the anomaly gate, the monitoring stack, and, most importantly, the leakage-aware training and model-selection protocol that produced the deployed forest in the first place. Doing this correctly is not simply a matter of calling an off-the-shelf converter. The common ONNX export path for tree ensembles goes through scikit-learn (`skl2onnx`): a Spark model has no scikit-learn twin, so that path would require re-fitting a new forest outside Spark, which breaks the claim that the served model is the one selected under the leakage-aware offline protocol (label-leaking `destination_port` removed, duplicate flows de-duplicated) rather than a freshly retrained substitute with its own, unaudited accuracy.

We instead read Spark’s own saved representation, node table and split thresholds stored as Parquet under the model directory, and translate it directly into the operators the ONNX tree-ensemble runtime expects, matching Spark’s split semantics, leaf-weighting, and probability normalization exactly rather than approximately. The same reader also drives a second, dependency-light export: a flattened NumPy node table, for Jetson/JetPack combinations where no prebuilt `onnxruntime` wheel exists. Both exports plug into the existing role-pipeline code behind one new interface, so switching backends is a one-line configuration change instead of a rewrite.

Contributions. (1) A swappable classifier-tier contract (`InferenceEngine`) with three backends – the original Spark `PipelineModel`, ONNX Runtime, and pure NumPy – selected by a single `EDGE_ENGINE` variable, integrated into the same Kafka role pipeline used by the companion deployment paper with no change to the anomaly gate or the monitoring stack; (2) A direct Spark-Parquet-to-ONNX exporter for `StandardScaler + RandomForestClassifier PipelineModels` that needs neither scikit-learn nor a retraining step, together with the equivalence argument (split direction, leaf-weight normalization, scaler mapping) that lets the exported graph reproduce Spark’s predictions rather than approximate them; (3) A pure-NumPy fallback exporter and vectorized forest-traversal engine for boards without an available `onnxruntime`

build; (4) An open, reproducible extension of the companion deployment’s code and benchmark protocol⁵.

2 Related Work

Edge intrusion detection. Single-board deployments of ML-based IDS are now common, from early Raspberry Pi systems [13] to recent ensemble and metaheuristic-optimized detectors on IoT-edge and industrial-control hardware [3, 12, 17] and autoencoder- or generative-model-based detection on Jetson boards [6, 4]. These studies optimize the detector; our companion deployment paper instead measured a distributed, streaming system (Kafka [7] plus a two-stage Spark pipeline [16, 10] across two Jetson Orin Nano Super boards) end to end, and found that the choice of *serving stack*, not the model, was the dominant cost once the anomaly gate had already removed most benign traffic. The present paper is scoped to closing exactly that gap.

Serving classical ML on constrained hardware. ONNX⁶ standardizes a small operator set, including `TreeEnsembleClassifier` for decision-tree ensembles, so a model trained in one framework can be served by any ONNX-compatible runtime; ONNX Runtime⁷ is the reference CPU/GPU implementation and is what we target. Nakandala et al. [11] make a related but different argument for tree ensembles specifically: they compile scikit-learn-style pipelines into tensor operations so a GPU tensor runtime (rather than a tree-optimized one) can serve them, and report large speedups over native scikit-learn on batched workloads. Their compiler, like the standard `skl2onnx` path, assumes a scikit-learn (or scikit-learn-compatible) source model; Spark’s `RandomForestClassificationModel` has no such representation, only its own saved Parquet node table, which is why we read that table directly instead of retraining through an intermediate framework. TensorRT⁸ takes the further step of compiling an ONNX graph for the Jetson’s GPU/DLA hardware; we target the ONNX Runtime CPU execution provider only and leave a GPU-accelerated path for future work, consistent with the companion paper’s own conclusion.

Evaluation protocol. Arp et al. [1] and Lanvin et al. [8] document how easily an offline security-ML evaluation can be inflated by leakage or dataset artifacts; our companion study responded by training on a leakage-aware, deduplicated CICIDS2017 [14] protocol and selecting a SHAP-explained [9] 30-feature Random Forest [2]. The present paper’s central design constraint follows directly from that: any serving path we add must reproduce that exact model’s predictions, not a re-fit approximation with its own untested accuracy, so *conversion correctness* is treated as a first-class requirement rather than an afterthought (Sect. 4).

⁵ https://github.com/vuthainguyen1602/Thesis_IDS_feature/numpy-prediction-engine.

(branch

⁶ <https://onnx.ai/>

⁷ <https://onnxruntime.ai/>

⁸ <https://developer.nvidia.com/tensorrt>

3 System Architecture

3.1 Swappable Classifier Tier

Fig. 1 extends the companion paper’s split-deployment architecture (host PC for Kafka/PostgreSQL/InfluxDB/Grafana, Jetson #1 as the anomaly gate, Jetson #2 as the classifier) with one change confined to Jetson #2: the classifier role now goes through an `InferenceEngine` contract with three interchangeable backends instead of a hard-coded PySpark call.

- `FeatureMatrixBuilder` (`edge/feature_matrix.py`) turns a batch of Kafka message dicts into a dense float32 matrix in a fixed column order, applying the same missing/NaN/Inf-to-zero policy as the existing Spark-side preprocessor. It imports no PySpark, so it is safe to use even when the classifier role never starts a JVM.
- `InferenceEngine` (`edge/inference_engine.py`) is the shared contract: every backend consumes that matrix and returns a `BatchPrediction` (predictions, per-row confidences, and timing/attack-count stats), so no engine-specific type (Spark `DataFrame`, ONNX tensor, NumPy array) leaks into the role-pipeline code. `create_inference_engine()` imports backends lazily, so selecting numpy or onnx never imports pyspark.
- `SparkInferenceEngine` wraps the original `PredictionEngine` unchanged, so the companion paper’s published benchmark path still runs exactly as measured; it exists purely as an adapter to the new contract.
- `OnnxInferenceEngine` and `NumpyInferenceEngine` load the exported artifacts described in Sect. 4 and never touch Spark.

The Jetson #2 role (`ClassifierPipeline` in `edge/role_pipelines.py`) only creates a `SparkSession` when `EDGE_ENGINE=spark`; for numpy or onnx the process never starts a JVM at all. Everything upstream of the classifier – the Kafka consumer, the anomaly gate on Jetson #1, the suspicious-flow forwarder, the InfluxDB/Grafana monitoring, and the PostgreSQL result sink – is untouched, and each stored verdict now records which backend produced it (`route: spark_classifier, numpy_classifier, or onnx_classifier`), so the three backends can be A/B-compared from the same replayed traffic.

3.2 Design Constraints

Two constraints shaped the contract. First, **no accuracy regression by construction**: the ONNX and NumPy backends are not independently trained models, they are re-encodings of the same fitted Spark forest, so any prediction difference is a conversion bug to fix, not a modeling trade-off to accept (Sect. 4 states the equivalence argument this relies on). Second, **no forced dependency**: a board that cannot install `onnxruntime` for its JetPack/Python combination – a real risk on ARM64 developer kits, where prebuilt wheels lag desktop platforms – can still shed the JVM by using `EDGE_ENGINE=numpy`, at the cost of the vectorized-but-unaccelerated inner loop described next.

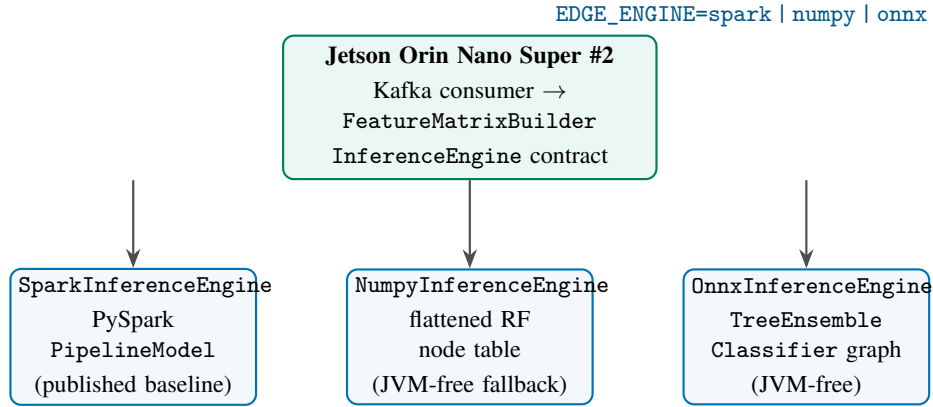


Fig. 1. Swappable classifier tier on Jetson #2: one feature-matrix builder, one `InferenceEngine` contract, three backends selected by a single environment variable. The gate on Jetson #1 and the Kafka/monitoring topology are unchanged from the companion paper.

4 Model Export and Conversion Correctness

4.1 Reading the Spark Model Directly

The deployed model is a Spark `PipelineModel` with three stages: `VectorAssembler`, `StandardScalerModel`, and `RandomForestClassificationModel`. Spark persists each stage’s parameters as JSON metadata plus, for the forest, a node table written as Parquet (one row per tree node, with `id`, `prediction`, `impurity`, `impurityStats`, `leftChild`, `rightChild`, `split`). `scripts/export_onnx.py` resolves the stage directories from the pipeline’s `stageUids` and reads this Parquet table with `pyarrow` alone, so the exporter needs neither a running Spark session nor `scikit-learn`: it can run on a laptop, in CI, or on the edge board itself.

4.2 Spark → ONNX Equivalence

Converting to ONNX’s `ai.onnx.ml.TreeEnsembleClassifier` operator requires matching three things exactly, not approximately:

- **Split direction.** Spark routes a continuous split left when `feature ≤ leftCategoriesOrThreshold[0]`; this maps directly onto `TreeEnsembleClassifier`’s `BRANCH_LEQ` mode with `trueNodeIds/falseNodeIds` set to Spark’s `leftChild/rightChild`, so no threshold or direction translation is needed beyond the field rename.
- **Leaf weighting.** Spark’s `predictRaw` sums each tree’s normalized leaf class distribution and `raw2probability` divides by the tree count, i.e. predictions are the *mean* of the trees’ class distributions. `TreeEnsembleClassifier` has no such aggregate mode (only `TreeEnsembleRegressor` exposes `aggregate_function`, and its classifier counterpart always sums class weights), so the per-tree averaging is folded into the exported weights themselves: each leaf’s `class_weights` is its normalized `impurityStats` distribution scaled by $w_t / \sum_t w_t$. Summing these across

trees reproduces Spark’s average, and `post_transform` is left at `NONE` because the scaling already sums to one.

- **Feature scaling.** Spark’s `StandardScaler` computes $(x - \text{mean}) \times (1/\text{std})$, with the scale forced to zero wherever the fitted std is zero (a constant feature); this is exactly the `ai.onnx.ml.Scaler` operator’s semantics, so the fitted offset/scale vectors are copied in as-is.

The exporter derives the output label with an `ArgMax` over the exported class scores rather than reusing Spark’s own label output field, since the two are mathematically identical for this model (the label is the arg-max class of the same averaged distribution) and `ArgMax` keeps the graph a strict function of the scores it also exposes, which simplifies the correctness check below. An optional `--validate-csv` flag runs the original Spark model and the exported ONNX graph on the same held-out rows and reports the prediction agreement rate; *[fill in the measured agreement rate and any max probability deviation once `export_onnx.py --validate` is run against a held-out CICIDS2017 sample].*

4.3 NumPy Fallback: A Flattened Forest

`scripts/export_numpy.py` reuses the same Parquet reader and produces a single `.npz` archive: the fitted scaler’s offset/scale vectors, and the whole forest flattened into parallel arrays (`tree_offsets`, `left`, `right`, `feature`, `threshold`, `is_leaf`, `leaf_scores`), with each tree’s leaf scores pre-weighted by $w_t / \sum_t w_t$ exactly as in the ONNX export, so both JVM-free backends encode the same equivalence argument. At inference time, `NumpyInferenceEngine` walks every tree with a vectorized loop over an active-row boolean mask: at each step, rows still active look up their current node’s split feature and threshold, move left or right, and rows that land on a leaf accumulate that leaf’s weighted class scores and drop out of the mask; the loop ends when no row is active in any tree. This keeps the whole batch’s traversal in NumPy array operations rather than a per-row Python loop, at the cost of walking every tree to its deepest active leaf in the batch rather than exploiting SIMD tree-specific kernels the way a compiled runtime (ONNX Runtime, or Hummingbird-style tensor compilation [11]) would; we treat it purely as a dependency-light fallback, not a competitor to the ONNX path on speed.

4.4 Reproducing Both Artifacts

`scripts/export_edge_artifacts.sh` runs both exporters from the one saved `PipelineModel` and feature-column list, producing `model/ids_rf.onnx` and `model/ids_rf_numpy.npz` in a single step, so a redeployment after retraining regenerates every serving artifact from the one source of truth.

5 Experimental Setup

The testbed is unchanged from the companion deployment paper: two NVIDIA Jetson Orin Nano Super Developer Kits (6-core Arm Cortex-A78AE, Ampere GPU, 8 GB

LPDDR5, 256 GB NVMe) and a host PC running Docker Compose (Kafka, PostgreSQL, InfluxDB, Grafana) over a LAN, replaying CICIDS2017 [14] flows through the same anomaly-gate-then-classifier topology. The classifier role (Jetson #2) is re-run under each of the three EDGE_ENGINE settings with the gate on Jetson #1 held fixed, so any throughput, latency, or energy difference is attributable to the classifier backend alone. *[State the CICIDS2017 replay rate, batch size, repetition count, and warmup policy used for the EDGE_ENGINE sweep once it is run; the companion paper’s protocol (100 flows/s, batch size 10, ≥ 3 repetitions with warmup excluded) is the default unless a different rate is needed to saturate the fastest backend.]* Reported metrics mirror the companion paper: verdict throughput (flows/s), p50/p95 latency (ms), per-node CPU/RAM/temperature, and energy per verdict (mJ, via tegrastats, idle baseline subtracted). We add one metric specific to this paper: the prediction-agreement rate between each JVM-free backend and the Spark model it was exported from, measured once offline (Sect. 4) and, where feasible, spot-checked online against the Spark backend’s verdicts on the same replayed flows.

6 Results

6.1 Single-Node Engine Cost (Reused from the Companion Microbenchmark)

The companion paper’s own single-node microbenchmark already reported the Spark side of this comparison: the deployed PySpark path reached 22.6 flows/s at a p95 of 532 ms, against 204.4 flows/s / 49.8 ms for the same forest under scikit-learn and 7,870.8 flows/s / 1.1 ms under ONNX Runtime – a $9\times$ and $348\times$ gap, respectively. We do not reproduce that Spark row here: Table 1 instead isolates the two JVM-free candidates from the same microbenchmark, to show how much of the remaining gap ONNX Runtime closes over the scikit-learn reference alone. ONNX Runtime reaches roughly $38\times$ scikit-learn’s throughput and about 2% of its energy per inference, before any pipeline integration.

Table 1. Single-node inference cost of the exported Random Forest under the two JVM-free candidate engines (companion microbenchmark, reproduced for reference).

Engine	Throughput (flows/s)	p95 latency (ms)	RAM (%)	Energy/inf. (mJ)
scikit-learn	204.4	49.8	13.3	2.91
ONNX Runtime	7870.8	1.1	15.7	0.06

Source: `papers/soict2026/results/benchmarks/benchmark_engines_results.json`, single-node baseline, batch size 10, 5,000 samples per engine. Both rows use the scikit-learn-exported reference forest, used in the companion paper to validate the conversion path prior to the direct Spark-Parquet export this paper describes (Sect. 4). The deployed PySpark path is not included as a row: its measured cost (22.6 flows/s, 532 ms p95) comes from a different measurement in the same study (the live classifier board, not this single-node JSON) and is cited in prose above instead of mixed into this table.

6.2 Conversion Correctness

[Report the Spark-vs-ONNX and Spark-vs-NumPy prediction agreement rate (target: 100% label agreement, near-zero probability deviation) from `export_onnx.py --validate-csv` on a held-out CICIDS2017 sample, once run.]

6.3 End-to-End Kafka Pipeline, Three Backends

[Report throughput, p50/p95 latency, per-node CPU/RAM/temperature, and energy per verdict for `EDGE_ENGINE=spark|numpy|onnx` on the live two-node Kafka pipeline, once the branch `feature/numpy-prediction-engine` is benchmarked on the Jetson cluster following the protocol in `papers/onnx2026/README.md`. Populate Table 2 and Fig. 2 from `papers/onnx2026/results/benchmarks/` once available; do not reuse Table 1’s single-node figures as a substitute, since Kafka consumption, batching, and JSON deserialization add pipeline overhead the single-node microbenchmark does not include.]

Table 2. End-to-end Kafka pipeline comparison on Jetson #2, three classifier backends, gate on Jetson #1 held fixed (mean $\pm$ std over repetitions).

EDGE_ENGINE	Throughput (flows/s)	Latency p95 (ms)	CPU (%)	Energy/verdict (mJ)
spark	[TBD]	[TBD]	[TBD]	[TBD]
numpy	[TBD]	[TBD]	[TBD]	[TBD]
onnx	[TBD]	[TBD]	[TBD]	[TBD]

[Engine-mode bars — plot once `papers/onnx2026/results/benchmarks/` has data]

Fig. 2. End-to-end Kafka pipeline comparison across classifier backends: verdict throughput (left) and p95 latency (right).

7 Discussion

The companion single-node microbenchmark already establishes that the JVM, not the model, is the dominant edge-serving cost for this system: the same forest is $9\times$ faster under scikit-learn and $348\times$ faster under ONNX Runtime than the deployed PySpark path,

and neither of those engines was, until this paper, wired into the streaming pipeline at all. Table 1 isolates the two JVM-free candidates and shows ONNX Runtime still reaching roughly $38\times$ scikit-learn’s own throughput, so compiling the forest to a tree-ensemble graph buys more than simply leaving the JVM behind. Closing that gap required more than pointing a converter at the model: because the deployed forest was selected under a leakage-aware protocol specifically to avoid the optimistic scores that leakage and dataset artifacts produce [1, 8], any export path that silently retrained or approximated the model would have undermined that selection. Reading Spark’s own Parquet node table directly, rather than going through a scikit-learn intermediate, is what lets us claim the ONNX and NumPy backends serve *the same* model rather than a plausible substitute – a claim Sect. 6 still needs to close with a measured agreement rate rather than the equivalence argument alone.

Limitations. At the time of writing, the swappable-engine code (branch `feature/numpy-prediction-engine`) has not yet been benchmarked end-to-end on the Jetson cluster; Table 2 and the conversion-agreement figures in Sect. 6 are placeholders pending that run, and the paper should not be treated as submission-ready until they are filled from real hardware. The NumPy backend’s vectorized-but-uncompiled tree traversal is expected to sit between scikit-learn and ONNX Runtime on throughput; it is included as a dependency-light fallback, not because it is expected to close the gap to ONNX Runtime on speed. The ONNX backend targets the CPU execution provider only; the Jetson’s GPU/Tensor cores, and a TensorRT compilation of the same graph, remain unexploited, consistent with the companion paper’s stated future work. Finally, both exports assume the deployed model stays a `StandardScaler + RandomForestClassifier PipelineModel`; a change of classifier family (e.g. to a gradient-boosted tree model) would require extending the Parquet reader, not just re-running the existing exporters.

8 Conclusion

We closed the gap the companion deployment paper identified but did not close: a JVM-hosted Spark serving path measured at $348\times$ the throughput of a matched ONNX Runtime engine, and no wiring from that measurement back into the running streaming pipeline. We built a swappable classifier-tier contract and two JVM-free backends – ONNX Runtime and a pure-NumPy fallback – both exported directly from Spark’s saved Parquet node tables rather than through a retrained scikit-learn intermediate, so the served model stays the one selected under the leakage-aware offline protocol. Both backends slot into the existing Kafka role pipeline behind a single `EDGE_ENGINE` switch, with the anomaly gate, monitoring stack, and topology from the companion paper unchanged. *[The remaining work is empirical: benchmark the three backends end-to-end on the Jetson cluster, measure Spark-vs-export prediction agreement on held-out data, and update Sect. 6 accordingly before this paper is submission-ready.]* Future work beyond that includes a TensorRT-compiled GPU path for the ONNX graph and re-running the anomaly gate itself through the same swappable contract.

Disclosure of Interests. The authors declare that they have no competing interests.

References

1. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 3971–3988 (2022)
2. Breiman, L.: Random forests. *Machine Learning* **45**(1), 5–32 (2001)
3. Chithra Rani, P.R., Baalaji, K.: Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms. *Discover Applied Sciences* **7**, 928 (2025). <https://doi.org/10.1007/s42452-025-06871-z>
4. Diaz-Gorriñ, J., Caballero-Gil, C., Brankovic, L.: Enhancing network security with generative AI on Jetson Orin Nano. *Applied Sciences* **16**(3), 1442 (2026). <https://doi.org/10.3390/app16031442>
5. Hamdan, S., Ayyash, M., Almajali, S.: Edge-computing architectures for internet of things applications: A survey. *Sensors* **20**(22), 6441 (2020). <https://doi.org/10.3390/s20226441>
6. Hasan, T., Hossain, A., Ansari, M.Q., Syed, T.H.: Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning (2025), arXiv:2501.15266
7. Kreps, J., Narkhede, N., Rao, J.: Kafka: a distributed messaging system for log processing. In: *Proceedings of the NetDB Workshop* (2011)
8. Lanvin, M., Gimenez, P.F., Han, Y., Majorczyk, F., Mé, L., Totel, É.: Errors in the CIDS2017 dataset and the significant differences in detection performances it makes. In: *Risks and Security of Internet and Systems (CRiSIS 2022), Revised Selected Papers*. pp. 18–33. *Lecture Notes in Computer Science*, Springer (2023). https://doi.org/10.1007/978-3-031-31108-6_2
9. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems* **30** (2017)
10. Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M.J., Zadeh, R., Zaharia, M., Talwalkar, A.: MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research* **17**(34), 1–7 (2016)
11. Nakandala, S., Saur, K., Yu, G.I., Karanasos, K., Curino, C., Weimer, M., Interlandi, M.: A tensor compiler for unified machine learning prediction serving. In: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association (2020)
12. Nasir, K., Badri, S.K., Alghazzawi, D.M., Alghamdi, M.Y., Alkhozai, M., Almakky, A., Alhazmi, R.M., Asghar, M.Z.: HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning. *Scientific Reports* **16**, 2313 (2026). <https://doi.org/10.1038/s41598-025-32183-8>
13. Sforzin, A., Gómez Mármol, F., Conti, M., Bohli, J.M.: RPiDS: Raspberry pi IDS — a fruitful intrusion detection system for IoT. In: *Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom)*. pp. 440–448 (2016). <https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCoM-IoP-SmartWorld.2016.0080>
14. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. pp. 108–116 (2018)
15. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. *IEEE Internet of Things Journal* **3**(5), 637–646 (2016). <https://doi.org/10.1109/JIOT.2016.2579198>
16. Zaharia, M., Chowdhury, M., Franklin, M.J., Shenker, S., Stoica, I.: Spark: Cluster computing with working sets. In: *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud'10)* (2010)

17. Zhang, Z., Zhang, L., Xu, J., Chen, Z., Wang, P.: An edge-deployable lightweight intrusion detection system for industrial control. *Electronics* **15**(3), 644 (2026). <https://doi.org/10.3390/electronics15030644>